Driver CAN Bus bas niveau (Bare-Metal CMSIS) pour microcontrôleur STM32L4

1. Fichier d'en-tête : can.h

```
#ifndef CAN_H
#define CAN_H

#include <stdbool.h>
#include <stm32l4xx.h>



#define CAN_ID_STD 0
#define CAN_ID_EXT 1

#define CAN_RTR_DATA 0
#define CAN_RTR_REMOTE 1

#define CAN_FILTER_SCALE_16BIT 0
#define CAN_FILTER_SCALE_32BIT 1

#define CAN_FILTER_MODE_MASK 0
#define CAN_FILTER_MODE_LIST 1

#define CAN_FILTER_FIFO0 0
#define CAN_FILTER_FIFO1 1

#define CAN_FILTER_ENABLE 1
#define CAN_FILTER_DISABLE 0

#define CAN_FIFO_RELEASE 1
#define CAN_FIFO_KEEP 0

#define CAN_LOOPBACK_ON 1
#define CAN_LOOPBACK_OFF 0

#define CAN_SILENT_ON 1
#define CAN_SILENT_OFF 0

#define CAN_TTCM_ENABLE 1
#define CAN_TTCM_DISABLE 0

#define CAN_ABOM_ENABLE 1
#define CAN_ABOM_DISABLE 0

#define CAN_AWUM_ENABLE 1
#define CAN_AWUM_DISABLE 0

#define CAN_NART_ENABLE 1
#define CAN_NART_DISABLE 0

#define CAN_RFLM_ENABLE 1
#define CAN_RFLM_DISABLE 0

#define CAN_TXFP_CHRONO 1
#define CAN_TXFP_ID_PRIO 0


typedef enum
{
    CAN1_PA11_12 = 0,
    CAN1_PB8_9,
    CAN1_PD0_1
} CAN_GPIO_Option;
```

```
int can_gpio_init(CAN_TypeDef *CANx, CAN_GPIO_Option option);


int can_init(CAN_TypeDef *CANx,
    bool TTCM, bool ABOM, bool AWUM, bool NART,
    bool RFLM, bool TXFP,
    uint32_t SJW, uint32_t BS1, uint32_t BS2,
    uint32_t PRESCALER,
    bool LOOPBACK, bool SILENT);


int can_transmit(CAN_TypeDef *CANx, uint32_t ID, bool EXT, bool RTR,
    uint8_t DLC, uint8_t *DATA);


void can_filter_init(CAN_TypeDef *CANx, uint32_t filter_number, bool scale_32bit,
    bool id_list_mode,
    uint32_t filter_register1, uint32_t filter_register2,
    uint32_t fifo, bool enable);


void can_receive(CAN_TypeDef *CANx, uint8_t fifo, bool release, uint32_t *id,
    bool *ext, bool *rtr,
    uint8_t *fmi, uint8_t *length, uint8_t *data);


#endif
```

2. Fichier d'implémentation : can.c

```
#include "can.h"



int can_gpio_init(CAN_TypeDef *CANx, CAN_GPIO_Option option)
{
    if (CANx != CAN1)
        return 1;

    switch (option)
    {
    case CAN1_PA11_12:
        // Enable GPIOA clock
        RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

        // Select Alternate Function mode (10)
        GPIOA->MODER &= ~(GPIO_MODER_MODE11 | GPIO_MODER_MODE12);
        GPIOA->MODER |= (GPIO_MODER_MODE11_1 | GPIO_MODER_MODE12_1);

        // Select high speed (10)
        GPIOA->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED11 | GPIO_OSPEEDR_OSPEED12);
        GPIOA->OSPEEDR |= (GPIO_OSPEEDR_OSPEED11_1 | GPIO_OSPEEDR_OSPEED12_1);

        // Select AF9 (CAN1) for PA11 and PA12
        GPIOA->AFR[1] &= ~(GPIO_AFRH_AFSEL11 | GPIO_AFRH_AFSEL12);
        GPIOA->AFR[1] |= (9 << GPIO_AFRH_AFSEL11_Pos) | (9 << GPIO_AFRH_AFSEL12_Pos);

        // PA11 Rx Pull UP (default = 1 recessive)
        GPIOA->PUPDR &= ~GPIO_PUPDR_PUPD11;
        GPIOA->PUPDR |= GPIO_PUPDR_PUPD11_0;

        // PA12 Tx Push-Pull
        GPIOA->OTYPER &= ~GPIO_OTYPER_OT12;

        break;

    case CAN1_PB8_9:
        // Enable GPIOB clock
        RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

        // Select Alternate Function mode (10)
```

```
        GPIOB->MODER &= ~(GPIO_MODER_MODE8 | GPIO_MODER_MODE9);
        GPIOB->MODER |= (GPIO_MODER_MODE8_1 | GPIO_MODER_MODE9_1);

        // Select High Speed (10)
        GPIOB->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED8 | GPIO_OSPEEDR_OSPEED9);
        GPIOB->OSPEEDR |= (GPIO_OSPEEDR_OSPEED8_1 | GPIO_OSPEEDR_OSPEED9_1);

        // Select AF9 (CAN1) for PB8 and PB9
        GPIOB->AFR[1] &= ~(GPIO_AFRH_AFSEL8 | GPIO_AFRH_AFSEL9);
        GPIOB->AFR[1] |= (9 << GPIO_AFRH_AFSEL8_Pos) | (9 << GPIO_AFRH_AFSEL9_Pos);

        // PB8 Rx Pull UP
        GPIOB->PUPDR &= ~GPIO_PUPDR_PUPD8;
        GPIOB->PUPDR |= GPIO_PUPDR_PUPD8_0;

        // PB9 Tx Push-Pull
        GPIOB->OTYPER &= ~GPIO_OTYPER_OT9;

        break;

    case CAN1_PD0_1:
        // Enable GPIOD clock
        RCC->AHB2ENR |= RCC_AHB2ENR_GPIODEN;

        // Select Alternate Function mode (10)
        GPIOD->MODER &= ~(GPIO_MODER_MODE0 | GPIO_MODER_MODE1);
        GPIOD->MODER |= (GPIO_MODER_MODE0_1 | GPIO_MODER_MODE1_1);

        // Select High Speed (10)
        GPIOD->OSPEEDR &= ~(GPIO_OSPEEDR_OSPEED0 | GPIO_OSPEEDR_OSPEED1);
        GPIOD->OSPEEDR |= (GPIO_OSPEEDR_OSPEED0_1 | GPIO_OSPEEDR_OSPEED1_1);

        // Select AF9 (CAN1) for PD0 and PD1
        // Note: Using AFR[0] (AFRL) for pins 0-7
        GPIOD->AFR[0] &= ~(GPIO_AFRL_AFSEL0 | GPIO_AFRL_AFSEL1);
        GPIOD->AFR[0] |= (9 << GPIO_AFRL_AFSEL0_Pos) | (9 << GPIO_AFRL_AFSEL1_Pos);

        // PD0 Rx Pull UP
        GPIOD->PUPDR &= ~GPIO_PUPDR_PUPD0;
        GPIOD->PUPDR |= GPIO_PUPDR_PUPD0_0;

        // PD1 Tx Push-Pull
        GPIOD->OTYPER &= ~GPIO_OTYPER_OT1;

        break;
    }

    // Enable CAN1 Clock
    RCC->APB1ENR1 |= RCC_APB1ENR1_CAN1EN;


    return 0;
}


int can_init(CAN_TypeDef *CANx, bool TTCM, bool ABOM, bool AWUM, bool NART,
    bool RFLM, bool TXFP, uint32_t SJW, uint32_t BS1, uint32_t BS2,
    uint32_t PRESCALER, bool LOOPBACK, bool SILENT)
{
    if (CANx != CAN1)
        return 1;

    // Exit sleep mode
    CANx->MCR &= ~CAN_MCR_SLEEP;

    // Enter initialization mode
    CANx->MCR |= CAN_MCR_INRQ;

    // Wait for hardware confirmation (INAK bit = 1, SLAK bit = 0)
    while ((CAN_MSR_INAK & CANx->MSR) == 0)
    {
```

```
}

// Time Triggered Communication Mode
if (TTCM)
{
    CANx->MCR |= CAN_MCR_TTCM;
}
else
{
    CANx->MCR &= ~CAN_MCR_TTCM;
}

// Automatic Bus-Off Management
if (ABOM)
{
    CANx->MCR |= CAN_MCR_ABOM;
}
else
{
    CANx->MCR &= ~CAN_MCR_ABOM;
}

// Automatic Wakeup Mode
if (AWUM)
{
    CANx->MCR |= CAN_MCR_AWUM;
}
else
{
    CANx->MCR &= ~CAN_MCR_AWUM;
}

// No Automatic Retransmission
if (NART)
{
    CANx->MCR |= CAN_MCR_NART;
}
else
{
    CANx->MCR &= ~CAN_MCR_NART;
}

// Receive FIFO Locked Mode
if (RFLM)
{
    CANx->MCR |= CAN_MCR_RFLM;  // Lock FIFO
}
else
{
    CANx->MCR &= ~CAN_MCR_RFLM; // Overwrite oldest message
}

// Transmit FIFO Priority
if (TXFP)
{
    CANx->MCR |= CAN_MCR_TXFP;  // FIFO priority
}
else
{
    CANx->MCR &= ~CAN_MCR_TXFP; // Priority by identifier
}

// Clear register
CANx->BTR = 0;

CANx->BTR |= (SJW - 1) << CAN_BTR_SJW_Pos;
CANx->BTR |= (BS2 - 1) << CAN_BTR_TS2_Pos;
CANx->BTR |= (BS1 - 1) << CAN_BTR_TS1_Pos;
CANx->BTR |= (PRESCALER - 1) << CAN_BTR_BRP_Pos;

if (LOOPBACK)
```

```
    {
        CANx->BTR |= (LOOPBACK << CAN_BTR_LBKM_Pos);
    }
    if (SILENT)
    {
        CANx->BTR |= (SILENT << CAN_BTR_SILM_Pos);
    }

    // Exit initialization mode to start CAN communication
    CANx->MCR &= ~CAN_MCR_INRQ;

    // Wait for normal mode confirmation (INAK bit = 0)
    while (CANx->MSR & CAN_MSR_INAK)
    {

    }

    return 0;
}

int can_transmit(CAN_TypeDef *CANx, uint32_t ID, bool EXT, bool RTR,
    uint8_t DLC, uint8_t *DATA)
{
    uint8_t mailbox;

    // Check for an empty mailbox
    if (CANx->TSR & CAN_TSR_TME0)
    {
        mailbox = 0;
    }
    else if (CANx->TSR & CAN_TSR_TME1)
    {
        mailbox = 1;
    }
    else if (CANx->TSR & CAN_TSR_TME2)
    {
        mailbox = 2;
    }
    else
    {
        return -1; // No mailbox available
    }

    // Clear the register
    CANx->sTxMailBox[mailbox].TIR = 0;

    if (EXT)
    {
        // Extended ID: Use 29 bits and set IDE bit
        CANx->sTxMailBox[mailbox].TIR |= (ID << 3) | CAN_TI0R_IDE;
    }
    else
    {
        // Standard ID: Use 11 bits
        CANx->sTxMailBox[mailbox].TIR |= (ID << 21);
    }

    // Set RTR bit if it is a Remote Frame
    if (RTR)
    {
        CANx->sTxMailBox[mailbox].TIR |= CAN_TI0R_RTR;
    }
    else
    {
        // Set Data Length Code (DLC)
        CANx->sTxMailBox[mailbox].TDTR &= ~CAN_TDT0R_DLC;
        CANx->sTxMailBox[mailbox].TDTR |= DLC;

        // Load Data bytes
        CANx->sTxMailBox[mailbox].TDLR =
            (DATA[0] << CAN_TDL0R_DATA0_Pos) | (DATA[1] << CAN_TDL0R_DATA1_Pos) |
```

```
            (DATA[2] << CAN_TDL0R_DATA2_Pos) | (DATA[3] << CAN_TDL0R_DATA3_Pos);
        CANx->sTxMailBox[mailbox].TDHR =
            (DATA[4] << CAN_TDH0R_DATA4_Pos) | (DATA[5] << CAN_TDH0R_DATA5_Pos) |
            (DATA[6] << CAN_TDH0R_DATA6_Pos) | (DATA[7] << CAN_TDH0R_DATA7_Pos);
    }

    // Request transmission
    CANx->sTxMailBox[mailbox].TIR |= CAN_TI0R_TXRQ;

    return mailbox;
}

void can_filter_init(CAN_TypeDef *CANx, uint32_t filter_number, bool scale_32bit,
    bool id_list_mode,
    uint32_t filter_register1, uint32_t filter_register2,
    uint32_t fifo, bool enable)
{
    // filter bank init
    CANx->FMR |= CAN_FMR_FINIT;

    // deactivate filter
    CANx->FA1R &= ~(1U << filter_number);

    // select scale
    if (scale_32bit)
        CANx->FS1R |= (1U << filter_number);
    else
        CANx->FS1R &= ~(1U << filter_number);

    if (id_list_mode)
        CANx->FM1R |= (1U << filter_number);
    else
        CANx->FM1R &= ~(1U << filter_number);

    CANx->sFilterRegister[filter_number].FR1 = filter_register1;
    CANx->sFilterRegister[filter_number].FR2 = filter_register2;

    // assign filter to FIFO
    switch (fifo)
    {
    case 0:
        CANx->FFA1R &= ~(1U << filter_number);
        break;
    case 1:
        CANx->FFA1R |= (1U << filter_number);
        break;
    }

    // activate filter
    if (enable)
        CANx->FA1R |= (1U << filter_number);

    // exit filter bank init mode
    CANx->FMR &= ~CAN_FMR_FINIT;
}


void can_receive(CAN_TypeDef *CANx, uint8_t fifo, bool release, uint32_t *id,
    bool *ext, bool *rtr,
    uint8_t *fmi, uint8_t *length, uint8_t *data)
{
    if ((CANx->sFIFOMailBox[fifo].RIR & CAN_RI0R_IDE) != 0) // if ext id
    {
        *ext = true;
        *id = (CANx->sFIFOMailBox[fifo].RIR >> CAN_RI0R_EXID_Pos);
    }
    else
    {
        *ext = false;
        *id = (CANx->sFIFOMailBox[fifo].RIR >> CAN_RI0R_STID_Pos);
    }
```

```
    *rtr = ((CANx->sFIFOMailBox[fifo].RIR & CAN_RI0R_RTR) != 0) ? 1 : 0;

    *length = (CANx->sFIFOMailBox[fifo].RDTR & CAN_RDT0R_DLC);

    *fmi = ((CANx->sFIFOMailBox[fifo].RDTR & CAN_RDT0R_FMI) >> CAN_RDT0R_FMI_Pos);

    if (*rtr == 0)
    {
        data[0] = (CANx->sFIFOMailBox[fifo].RDLR & CAN_RDL0R_DATA0);
        data[1] = ((CANx->sFIFOMailBox[fifo].RDLR & CAN_RDL0R_DATA1) >> CAN_RDL0R_DATA1_Pos);
        data[2] = ((CANx->sFIFOMailBox[fifo].RDLR & CAN_RDL0R_DATA2) >> CAN_RDL0R_DATA2_Pos);
        data[3] = ((CANx->sFIFOMailBox[fifo].RDLR & CAN_RDL0R_DATA3) >> CAN_RDL0R_DATA3_Pos);

        data[4] = ((CANx->sFIFOMailBox[fifo].RDHR & CAN_RDH0R_DATA4) >> CAN_RDH0R_DATA4_Pos);
        data[5] = ((CANx->sFIFOMailBox[fifo].RDHR & CAN_RDH0R_DATA5) >> CAN_RDH0R_DATA5_Pos);
        data[6] = ((CANx->sFIFOMailBox[fifo].RDHR & CAN_RDH0R_DATA6) >> CAN_RDH0R_DATA6_Pos);
        data[7] = ((CANx->sFIFOMailBox[fifo].RDHR & CAN_RDH0R_DATA7) >> CAN_RDH0R_DATA7_Pos);
    }

    // release fifo mailbox if true
    if (release)
        switch (fifo)
        {
        case 0:
            CANx->RF0R |= CAN_RF0R_RFOM0;
            break;
        case 1:
            CANx->RF1R |= CAN_RF1R_RFOM1;
            break;
        }
}
```

3. Programme Principal : main.c

```
#include <stm32l4xx.h>
#include "can.h"

#define VERSION 32

void delay(uint32_t delay_time);


int main(void)
{
    uint32_t id = 0;
    bool ext = 0;
    bool rtr = 0;
    uint8_t fmi = 0;
    uint8_t length = 0;
    uint8_t data_received[8] = {0};

    CAN_GPIO_Option opt = CAN1_PB8_9;

    can_gpio_init(CAN1, opt);
    /* clock = 4MHz, baud rate = 250 kHz
       clock / baud rate = 16
       on peut choisir prescaler = 2 et total tq = 8
       ou
       prescaler = 1 et total tq = 16
       pour bs1 :
       (1 + bs1) / total tq (= 8) = 87.5
       => bs1 = 6
       bs2 = total tq - bs1 - sync = 1
    */
    can_init(CAN1, CAN_TTCM_DISABLE, CAN_ABOM_DISABLE, CAN_AWUM_DISABLE,
        CAN_NART_DISABLE, CAN_RFLM_DISABLE,
        CAN_TXFP_ID_PRIO, 1, 6, 1, 2, CAN_LOOPBACK_ON, CAN_SILENT_OFF);

    uint8_t DATA[4];
    DATA[0] = 'E';
```

```
    DATA[1] = 'N';
    DATA[2] = 'S';
    DATA[3] = 'A';

#if VERSION == 32

    can_filter_init(CAN1, 0, CAN_FILTER_SCALE_32BIT, CAN_FILTER_MODE_MASK,
        0, 0, CAN_FILTER_FIFO0, CAN_FILTER_ENABLE);

#elif VERSION == 16

    can_filter_init(CAN1, 0, CAN_FILTER_SCALE_16BIT, CAN_FILTER_MODE_MASK,
        0, 0, CAN_FILTER_FIFO0, CAN_FILTER_ENABLE);

#endif

    can_transmit(CAN1, 0x22, CAN_ID_STD, CAN_RTR_DATA, 4, DATA);

    // wait until message is pending in FIFO
    while ((CAN1->RF0R & CAN_RF0R_FMP0) == 0)
    {

    }

    can_receive(CAN1, 0, CAN_FIFO_RELEASE, &id, &ext, &rtr, &fmi, &length,
        data_received);

    // turn on led if message is received
    // configuring built-in led
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~GPIO_MODER_MODE5;
    GPIOA->MODER |= GPIO_MODER_MODE5_0;
    // led off
    GPIOA->ODR &= ~GPIO_ODR_OD5;

    if ((DATA[0] == data_received[0]) && (DATA[1] == data_received[1])
        && (DATA[2] == data_received[2]) && (DATA[3] == data_received[3]))
        // led on
        GPIOA->ODR |= GPIO_ODR_OD5;

    while (1)
    {
        // delay(20000);
    }
}

void delay(uint32_t delay_time)
{
    for (volatile uint32_t i = 0; i < delay_time; i++)
    {

    }
}
```